



PROJETO

multidisciplinar

Miriã, Thiago, Yasmin, Raul, Luiz e Luciano

IFMS 2026

Linguagem de programação 3, Banco de Dados e
Engenharia de Software





RELATÓRIOS E TRABALHO EM EQUIPE

- Relatório de Sprints e Relatório de Caso de Uso
 - Grupo de Whatzzapp
 - Separação de tarefas semanais
 - Colaboração coletiva
- 

RAUL-

BANCO DE DADOS E API

LUIZ- BANCO DE DADOS E GUTHUB

```
CREATE TABLE aluno (  
  id_al SERIAL PRIMARY KEY,  
  nome VARCHAR(100),  
  telefone VARCHAR(20),  
  condicao_aluno VARCHAR(20)  
);
```

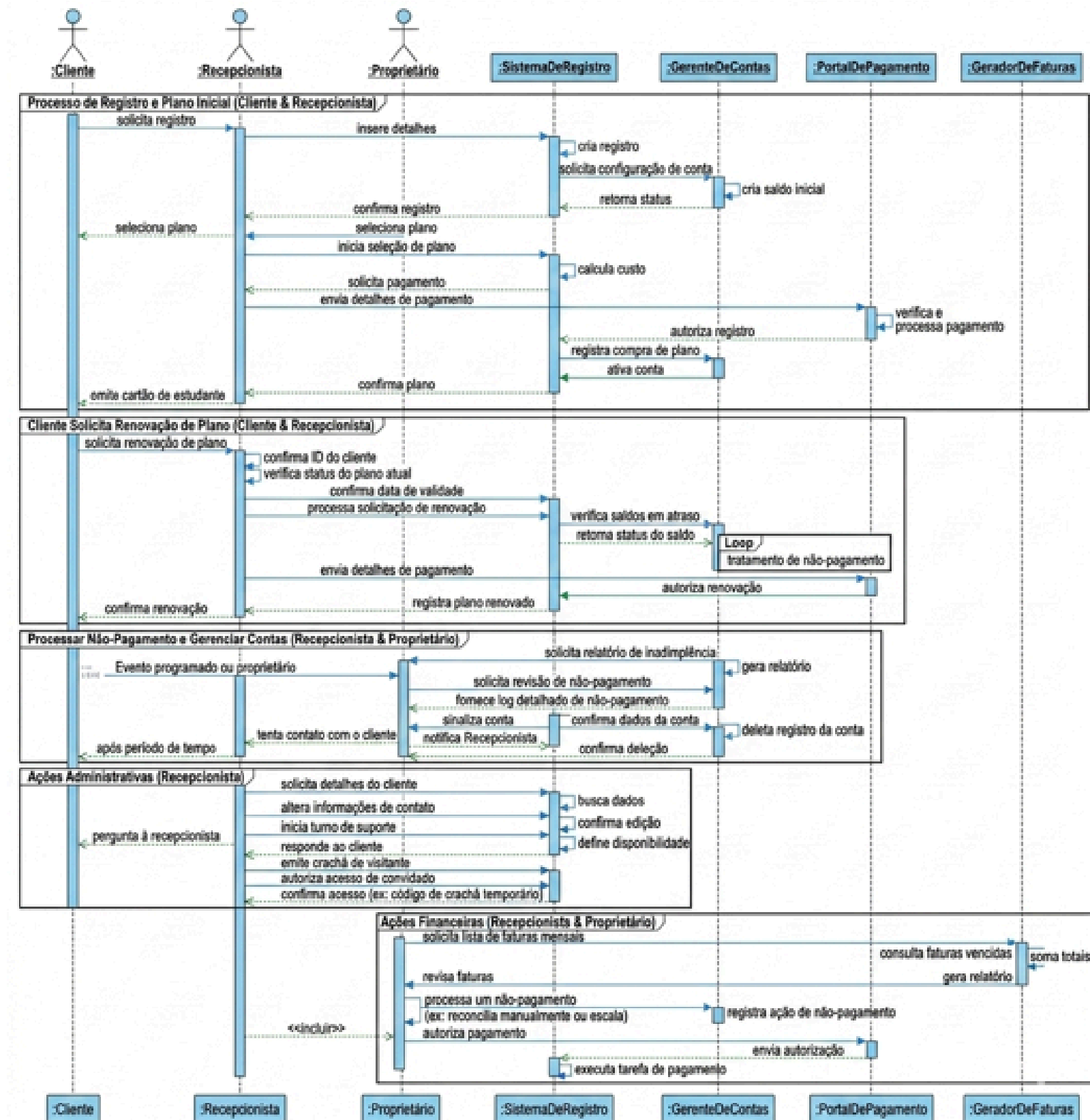
```
CREATE TABLE plano (  
  id_pla SERIAL PRIMARY KEY,  
  nome VARCHAR(50),  
  valor DECIMAL(10,2)  
);
```

```
CREATE TABLE pagamento (  
  id_pag SERIAL PRIMARY KEY,  
  id_al INT,  
  id_pla INT,  
  valor DECIMAL(10,2),  
  condicao_pagamento VARCHAR(20),  
  data_pagamento DATE,  
  hora_pagamento TIME,
```

```
FOREIGN KEY (id_al) REFERENCES  
  aluno(id_al),  
  FOREIGN KEY (id_pla)  
  REFERENCES plano(id_pla)  
);
```

MIRIÃ- ENGENHARIA DE SOFTWARE

4.1 Diagrama de Sequência 1



```
7 //GET
8 routes.get('/', async (req, res) => {
9     const result = await db.query('SELECT * FROM aluno');
10
11     res.status(200).json(result.rows);
12 });
```

GET

POST

```
14 //POST
15 routes.post('/', async (req, res) => {
16
17     const {nome, telefone, condicao_aluno} = req.body;
18
19     if(!nome || !telefone || !condicao_aluno){
20         return res.status(400).json({mensagem: 'Campos não preenchidos'});
21     }
22
23     const sql = `
24         INSERT INTO aluno (nome, telefone, condicao_aluno)
25         VALUES ($1, $2, $3)
26         RETURNING *
27     `;
28
29     const valores = [nome, telefone, condicao_aluno];
30
31     const result = await db.query(sql, valores);
32
33     res.status(201).json(result.rows[0]);
34 });
```



```
36 //PUT
37 routes.put('/:id_al', async (req, res) => {
38
39     const {id_al} = req.params;
40
41     if(!id_al){
42         return res.status(400).json({mensagem: 'id do aluno é obrigatório'});
43     }
44
45     const {nome, telefone, condicao_aluno} = req.body;
46
47     if(!nome || !telefone || !condicao_aluno){
48         return res.status(400).json({mensagem: 'Campos não preenchidos'});
49     }
50
51     const sql = `
52         UPDATE aluno
53         SET nome = $1 , telefone = $2 , condicao_aluno = $3
54         WHERE id_al = $4
55         RETURNING *
56     `;
57
58     const valores = [nome, telefone, condicao_aluno, id_al];
59
60     const result = await db.query(sql, valores);
61
62     if(result.rows.length === 0){
63         return res.status(404).json({mensagem: 'Aluno não encontrado.'});
64     }
65
66     res.status(200).json(result.rows[0]);
67 });
```

DELETE

PUT

```
69 //DELETE
70 routes.delete('/:id_al', async (req, res) => {
71
72     const {id_al} = req.params;
73
74     if(!id_al){
75         return res.status(400).json({mensagem: 'O id do aluno é obrigatório'});
76     }
77
78     const sql = `
79         DELETE FROM aluno
80         WHERE id_al = $1
81         RETURNING *
82     `;
83
84     const valores = [id_al];
85
86     const result = await db.query(sql, valores);
87
88     if(result.rows.length === 0) {
89         return res.status(404).json({mensagem: 'Aluno não encontrado.'});
90     }
91
92     res.status(200).json({mensagem: `Aluno com ID ${id_al} foi excluído com sucesso.`});
93 });
```

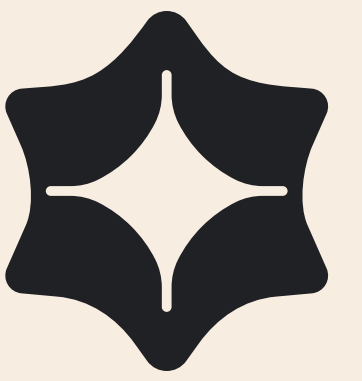
JOIN

```
34 //GROUP BY
35 routes.get('/app', async (req, res) => {
36
37     const result = await db.query(`
38         SELECT
39             pl.nome AS plano,
40             SUM(p.valor) AS total_arrecadado
41         FROM pagamento p
42         INNER JOIN plano pl ON p.id_pla = pl.id_pla
43         GROUP BY pl.nome
44     `);
45
46     res.status(200).json(result.rows);
47 });
```

GROUP

```
13 //JOIN
14 routes.get('/relatorio', async (req, res) => {
15
16     const result = await db.query(`
17         SELECT
18             p.id_pag,
19             a.nome AS aluno,
20             a.telefone,
21             pl.nome AS plano,
22             p.valor,
23             p.condicao_pagamento,
24             p.data_pagamento,
25             p.hora_pagamento
26         FROM pagamento p
27         INNER JOIN aluno a ON p.id_al = a.id_al
28         INNER JOIN plano pl ON p.id_pla = pl.id_pla
29     `);
30
31     res.status(200).json(result.rows);
32 });
```

Luciano - front End



O Front-End é a parte visual de um site ou aplicação, responsável pela interação com o usuário.

HTML

- Estrutura o conteúdo da página.
- Define elementos como títulos, textos, imagens e links.

Style.css

- Responsável pelo design e aparência.
- Controla cores, fontes, espaçamento e layout.

JavaScript

- Adiciona interatividade e dinamismo.
- Permite criar animações, validações de formulários e respostas a ações do usuário.



CONCLUSÃO



CONCLUSÃO



Trabalhamos em equipe



desenvolvemos um servidor



ajudamos uns aos outros





OBRIGADA